

Christmas Wishlist App — Project Documentation

Autor: Christmas Wishlist Team | Data: 10/08/2026

Christmas Wishlist App — Project Documentation

A family-focused web application built with Next.js that lets each member log in with their face, build a personal wishlist with photos and links, and let other members privately reserve gifts.

1. Project Overview

The Christmas Wishlist App is a modern, mobile-friendly web application where family members can:

- Log in using **facial recognition** (front-facing camera only).
- Manage a personal wishlist of items, each with a name, size, optional purchase link, and an optional photo (taken with the camera or uploaded).
- **Reserve gifts** on other members' wishlists, with privacy: the wishlist owner never sees who reserved their items.
- Browse a **members directory** with search, favorites, and a grid/list view toggle.
- Update their own **login (reference) photo** from the dashboard.

The app uses a "demo-grade" authentication model: face matching runs entirely on the client, which is acceptable for a family/personal app but not a hardened authentication system.

2. Technology Stack

Area	Technology
-----	-----
Framework	Next.js 16.3.0 (App Router, Turbopack build)
UI Library	React 19.2.8
Language	TypeScript 5
Styling	Tailwind CSS 4
Backend	Supabase (Postgres + Storage + REST API)
Face AI	@vladmandic/face-api 1.7.15 (SSD Mobilenet v1 + landmarks + recognition)
Icons	lucide-react
Tooling	ESLint 9, npm scripts (dev , build , start , lint)

3. Architecture & Data Flow

The app follows a **client-side (browser) architecture**:

1. The user opens the login page, which loads the face-api machine learning models from `public/models`.
2. On login, the browser fetches all users and their reference photos, computes a face descriptor for each reference photo, and caches them.
3. When the user clicks "Recognize me", the app captures a frame from the webcam and computes its face descriptor.
4. The captured descriptor is compared (euclidean distance) against all cached references; the closest match under the threshold is selected as the logged-in user.
5. After login, the app stores a lightweight session (user id + name) and shows the dashboard.
6. The dashboard reads/writes data through the Supabase REST API using the anon key.

Why client-side? Face-api.js and its bundled TensorFlow.js are browser-only, so all recognition logic runs in the browser. The server only serves static assets and never evaluates face-api code (it is dynamically imported to avoid SSR issues).

4. Project Structure

% CODE

```
app/
  layout.tsx          Global layout (fonts, providers)
  page.tsx            Landing page
  login/page.tsx      Login screen (face recognition)
  dashboard/page.tsx  Main dashboard (own wishlist + directory)
  wishlist/[userId]/page.tsx Member wishlist view (reserve items)

components/
  AppHeader.tsx       Sticky header with log out
  AuthProvider.tsx    React context for the logged-in user
  Avatar.tsx          Initials/photo avatar
  CameraCapture.tsx   Reusable webcam component (mirror + facing mode)
  FaceAuth.tsx        Face login flow
  FacePhotoModal.tsx  Update-your-login-photo modal (front camera)
  ItemModal.tsx       Add/edit item form (name, size, link, photo)
  MyWishlistCard.tsx  Own wishlist: add/edit/delete items, thumbnails
  UserDirectory.tsx   Searchable members directory (grid/list toggle)
  WishlistViewer.tsx  Other member's wishlist with reserve buttons
  LuckyOneCard.tsx    "Lucky one" banner + reveal card

lib/
  supabase.ts         Supabase client factory (anon key)
  types.ts            TypeScript interfaces (User, WishlistItem, ...)
  data.ts             Data access layer (fetch/insert/update/delete)
  session.ts          Session persistence (localStorage)
  face.ts             Face models, descriptor, distance, image loading
  recognition.ts       Reference building + identify user (threshold 0.45)
  faceImage.ts        Upload/resize of login reference photos
  itemImage.ts        Resize + upload/delete of item photos

public/
  models/             face-api model weights (SSD, landmarks, recognition)
  faces/              Reference photos per member (face_recognition_id.jpg)

supabase/
  schema.sql          Full schema + seed roster
  add-item-images.sql Migration: item photo column + bucket
  add-face-update.sql Migration: face photo column + bucket
```

5. Authentication — Face Recognition

5.1 Flow

1. Models load once (`lib/face.ts`, `loadFaceModels`).
2. `buildReferences(users)` in `lib/recognition.ts` loads each member's photo and computes a 128-dim descriptor. References are cached; the cache key includes the user's face image URL so it invalidates when a photo changes.
3. `identifyUser(users, frame)` captures a descriptor and finds the closest reference.

5.2 Tuning Values

- **Detection minConfidence = 0.5** (`lib/face.ts`): low-confidence boxes (chairs, objects, empty scenes) are rejected so an empty scene reports "no face" instead of a garbage descriptor.
- **MATCH_THRESHOLD = 0.45** (`lib/recognition.ts`): the closest cross-person distance among all 17 references is ~0.453 (Gianna !" Joy), so 0.45 leaves a small but real safety margin. Lower it (e.g., 0.42) if mis-identifications ever appear.
- Noise and solid-color frames correctly produce **no face** at both `mc=0.5` and `mc=0.3`.

5.3 Reference Photos

- Each user has a `face_recognition_id` (e.g., `papa_jojo`) that maps to a photo at `public/faces/<id>.jpg`.
- If the user has a `face_image_url` (updated from the dashboard), that URL is used instead.
- Photos are served with CORS `*` so the browser can read pixels into canvas for descriptor computation.

5.4 Updating the Login Photo

From the dashboard, "Update face photo" opens a modal with the **front-facing camera only**. The captured photo is resized (max 800px JPEG), uploaded with upsert to the ``faces`` storage bucket as ``<face_recognition_id>.jpg``, and the ``users.face_image_url`` (and ``profile_picture``) is updated. The change takes effect on the **next login** because the recognition cache automatically invalidates.

6. Data Model (Supabase)

6.1 users

```
%¶ CODE
id                uuid          primary key
profile_picture   text
face_image_url    text          (optional, overrides local reference photo)
created_at        timestampz    default now()
```

6.2 wishlist_items

```
%¶ CODE
id                uuid          primary key
owner_id          uuid          references users(id) on delete cascade
purchase_link     text
image_url         text          (public URL of the item photo)
created_at        timestampz    default now()
updated_at        timestampz    default now()
```

6.3 reservations

```
%¶ CODE
id                uuid          primary key
wishlist_item_id  uuid          references wishlist_items(id) on delete cascade
reserved_by_user_id  uuid        references users(id) on delete cascade
created_at        timestampz    default now()
unique (wishlist_item_id, reserved_by_user_id)
```

6.4 Database guards

- **enforce_single_gift_reservation** (before-insert trigger): a single-gift item (`allow_multiple = false`) may only ever have one reservation.
- **trim_extra_reservations(p_item_id)**: keeps only the earliest reservation when an owner flips an item back to single-gift.

6.5 Storage buckets

Bucket	Public Policies (anon)	Purpose	
--------	--------------------------	---------	--

-----	-----	-----	-----
item-images	yes	select, insert, delete	Item photos (camera/upload)
faces	yes	select, insert, update, delete	Login reference photos

Item photos are uploaded to `item-images/<ownerId>/<uuid>.jpg`; face photos to `faces/<face_recognition_id>.jpg` (upsert).

7. Features

7.1 Item Management

Members add/edit/delete items from "My Wishlist". Each item has:

- Item name and size (required).
- Purchase link (optional).
- Photo (optional): taken with the camera (**switchable front/back camera**) or uploaded from the device. Photos are resized to "d800px JPEG client-side before upload.
- "Allow multiple gifts" toggle so multiple people can reserve the same item.

Item rows show a thumbnail (click to view full size). Replacing/removing a photo deletes the old stored image, and deleting an item also deletes its stored image.

7.2 Reservations (gift claiming)

- A member can reserve/unreserve items on other members' wishlists.
- Single-gift items show **Reserved** once claimed; multi-gift items can be reserved by several people.
- **Privacy**: reservation names are never shown to the owner or other members — only the current user sees their own "Reserved by You" state.

7.3 Members Directory

- Lists all members except yourself, with avatars and a **3-item preview** + "See more" link.
- **Favorites**: heart button per member (stored in `favorite_user_id`).
- **Grid/list toggle**: grid shows 2–6 columns responsively; list shows stacked full-width cards.
- **Search** to filter members by name.

7.4 Lucky One

- A member flagged as `lucky_one` sees a banner; other members with `see_lucky_one = true` can see a card revealing who the lucky one is.

7.5 Item Photos (Camera + Upload)

The item form provides both **Take photo** (inline camera with a front/back switch overlay button) and **Upload** (`<input type="file" accept="image/*">`), with a live preview and Replace/Remove controls in edit mode.

8. Environment & Configuration

Create `.env.local`:

```
%T CODE
NEXT_PUBLIC_SUPABASE_ANON_KEY=<anon key>
```

The Supabase project for this app also requires running the SQL migrations in `supabase/schema.sql`, `supabase/add-item-images.sql`, and `supabase/add-face-update.sql` in the Supabase SQL Editor (the anon key cannot create columns, buckets, or policies).

The `opencode.json` at the project root registers the `document-generator` MCP server (`npx -y document-generator-mcp@latest`) used to produce this documentation.

9. Commands

```
%T BASH
npm install          # install dependencies
npm run dev          # start development server
npm run build        # production build (type-check included)
npm run start        # run the production build
npm run lint         # ESLint
```

10. Member Roster

17 members with reference photos in `public/faces/` and face login enabled:

Aaron, Archie, Mandy, Edder, Mikan, Mama Arcy, Papa Jun, Joy, Lin, Michael, Gianna, Papa Jojo, Mama Rosebie, Joana, Mark, Andrew, Angel.

All wishlists start empty; each member adds items from their dashboard.

11. Deployment Notes

- This is a **static-ish app** with one dynamic route (`/wishlist/[userId]`); it deploys fine on Vercel.
- Set the two `NEXT_PUBLIC_*` env vars in the host platform (Vercel project settings) — they are required at build/runtime.
- Face models and reference photos ship in `public/` and are served statically.
- Camera features require ``localhost`` or **HTTPS** (browser `getUserMedia` constraint).
- Storage buckets are public, so any image URL is viewable by anyone who has it — acceptable for a private family app, not for sensitive data.

12. Known Notes & Limitations

- **Demo-grade auth**: face matching runs client-side and a printed photo can spoof a match. Fine for family use, not for security.
- **Threshold margin**: closest cross-person distance is ~ 0.453 vs a 0.45 match threshold — works today, but lower the threshold slightly if mis-identifications occur.

- Requires a modern browser with a camera (Chrome/Edge/Firefox/Safari).
- Session is stored in localStorage and is not encrypted.

Generated by the Christmas Wishlist project team using the document-generator MCP server.